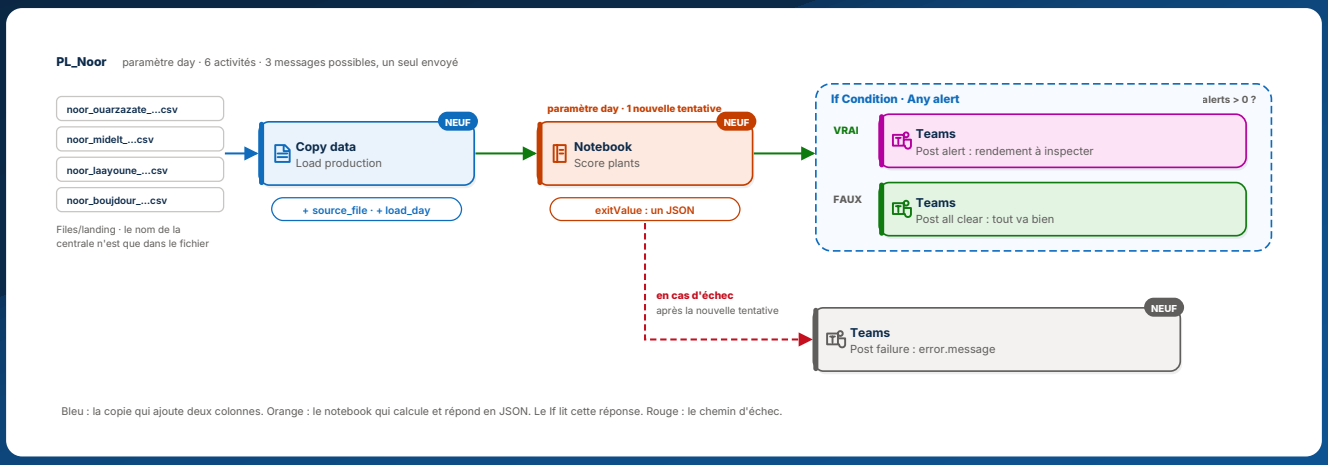
MICROSOFT FABRIC · DATA FACTORY · NOTEBOOK · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : une copie qui étiquette, un notebook qui répond

CHARGER · ÉTIQUETER · CALCULER · RÉPONDRE · DÉCIDER · PRÉVENIR

Maroc, mercredi 21 octobre 2026. Quatre centrales solaires Noor envoient leur production heure par heure. Une seule copie charge les quatre fichiers et grave l'origine dans chaque ligne. Un notebook paramétré classe les centrales et répond en JSON. Le pipeline lit la réponse, choisit son message, et prévient aussi quand tout casse.



6	96	3 370	1	3
activités	relevés horaires	MWh produits	centrale en alerte	chemins de sortie

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS
Un espace de travail Fabric sur une capacité, Microsoft Teams avec une équipe

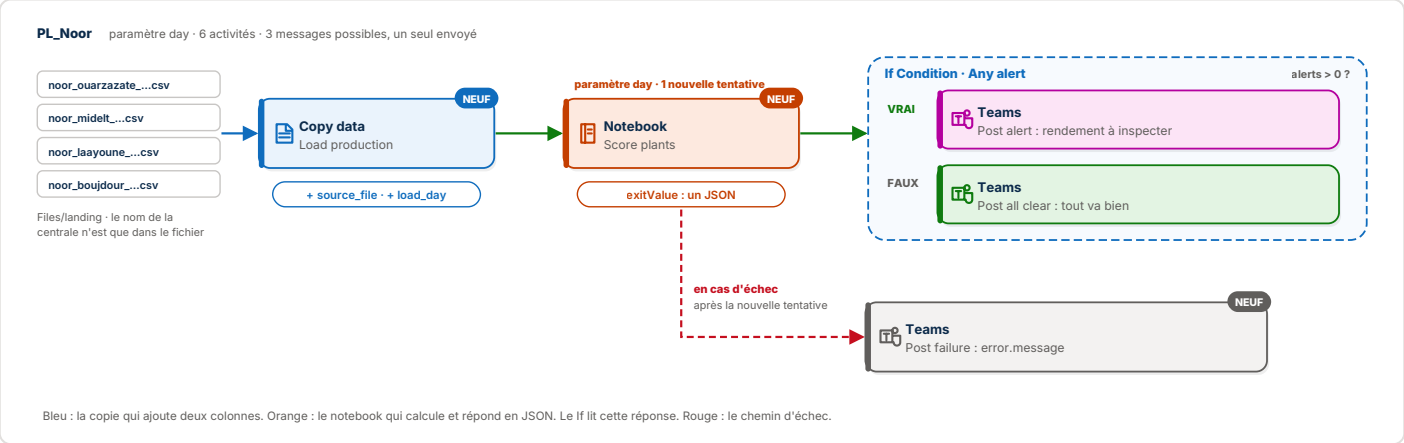
FICHIERS DU PROJET
Quatre CSV de production, NB_Noor.ipynb, expressions.txt

DURÉE
Environ 30 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que vous allez construire

Le Maroc produit une grande partie de son électricité solaire dans les centrales Noor. Je vous propose de suivre quatre d'entre elles le 21 octobre 2026 : chacune envoie un CSV de 24 lignes avec `ts`, `ghi_wm2` (le soleil reçu) et `mwh` (l'énergie produite). Petit piège réaliste : le nom de la centrale n'est écrit **que dans le nom du fichier**. Dans ce pipeline, je vous montre cinq réflexes de professionnel : **étiqueter chaque ligne avec son fichier d'origine**, **passer un paramètre à un notebook**, **récupérer sa réponse en JSON**, **réessayer automatiquement** et **prévenir avec la cause exacte quand ça casse**.



Le pipeline en une image : un laboratoire d'analyses

L'accueil
La copie reçoit tous les tubes d'un coup et colle sur chacun l'étiquette de son patient.

L'analyste
Le notebook reçoit la date à analyser et calcule les résultats.

La fiche
exitValue : un résumé JSON rendu au pipeline.

Le médecin
La condition Si lit la fiche et choisit le message.

L'alarme
La flèche rouge prévient si l'analyse échoue, avec la cause.

Les activités

ACTIVITÉ	TYPE	RÔLE
`Load production`	Copy data	Charge les 4 CSV en une fois dans `production_raw`, et ajoute `source_file` et `load_day` à chaque ligne
`Score plants`	Notebook	Reçoit `day`, classe les centrales, écrit `plant_scores`, renvoie un JSON
`Any alert`	If Condition	Lit le JSON : au moins une centrale en alerte ?
`Post alert` · `Post all clear`	Microsoft Teams	Un seul des deux part, selon la réponse du notebook
`Post failure`	Microsoft Teams	Ne part que si le notebook échoue, avec la cause exacte

FACTEUR DE CHARGE L'énergie du jour divisée par ce que la centrale produirait à pleine puissance pendant 24 h. Un bon parc solaire fait 15 à 25 %.

RENDEMENT (PR) L'énergie produite divisée par ce que le soleil reçu permettait. Sous 0,75, les panneaux perdent quelque chose : ici, le sable à Boujdour.

ÉTAPE 1

Le lakehouse et le notebook qui répond

1 · OBJETS | 2 · COPIER ET ÉTIQUETER | 3 · DÉCIDER | 4 · LE CHEMIN D'ÉCHEC

1 LH_Noor — LAKEHOUSE

1. Dans votre espace de travail, **Nouvel élément**, **Lakehouse**, nom `LH_Noor`.
2. Dans **Fichiers**, **Nouveau sous-dossier** : `landing`.
3. Dans `landing`, **Charger**, **Charger des fichiers** : les **quatre** CSV, en une fois.

2 NB_Noor — NOTEBOOK IMPORTÉ

1. Dans l'espace de travail, **Importer**, **Notebook**, **À partir de cet ordinateur** : `NB_Noor.ipynb`.
2. Ouvrez-le. Explorateur à gauche, **Ajouter des sources de données** (ou **Lakehouses**, **Ajouter**) : `LH_Noor`.
3. Survolez la cellule `day = "2026-10-21"` : l'étiquette **Cellule de paramètres** doit s'afficher. Sinon, **…** de la cellule, **Activer/désactiver la cellule de paramètre**.
4. Ne l'exécutez pas : `production_raw` n'existe pas encore.

```
# NB_Noor : classe les centrales solaires du jour et renvoie un
verdict au pipeline

# ===== Cellule 1 : cellule de paramètres (le pipeline remplace
cette valeur) =====
day = "2026-10-21"

# ===== Cellule 2 : le calcul =====
import json, re
from pyspark.sql import functions as F

if not re.fullmatch(r"\d{4}-\d{2}-\d{2}", day):
    raise ValueError(f"Date invalide reçue du pipeline :
{day}")

# capacité retenue pour l'exercice, en MW
PLANTS = {"noor_ouarzazate": ("Noor Ouarzazate", 72),
"noor_midelt": ("Noor Midelt", 600),
          "noor_laayoune": ("Noor Laâyoune", 85),
"noor_boujdour": ("Noor Boujdour", 20)}
ref = spark.createDataFrame([(k, n, float(c)) for k, (n, c) in
PLANTS.items()], "code string, plant string, capacity_mw
double")

raw = (spark.read.table("production_raw")
       .filter(F.col("load_day") == day)
       .withColumn("code", F.regexp_extract("source_file",
r"(noor_[a-z]+)", 1))
       .withColumn("mwh", F.col("mwh").cast("double"))
       .withColumn("ghi", F.col("ghi_wm2").cast("double")))

scores = (raw.groupBy("code")
          .agg(F.round(F.sum("mwh"), 1).alias("mwh"),
F.sum("ghi").alias("ghi_wh"))
          .join(ref, "code")
          .withColumn("capacity_factor_pct", F.round(100 *
F.col("mwh") / (F.col("capacity_mw") * 24), 1))
          .withColumn("performance_ratio",
F.round(F.col("mwh") / (F.col("capacity_mw") *
F.col("ghi_wh") / 1000), 2))
          .withColumn("status",
F.when(F.col("performance_ratio") < 0.75,
"alerte").otherwise("ok"))
          .withColumn("day", F.lit(day))
          .select("day", "plant", "capacity_mw", "mwh",
"capacity_factor_pct", "performance_ratio", "status"))
scores.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("plant_scores")

rows = scores.orderBy(F.desc("capacity_factor_pct")).collect()
alerts = [r["plant"] for r in rows if r["status"] == "alerte"]
verdict = {
    "day": day,
    "plants": len(rows),
    "total_mwh": round(sum(r["mwh"] for r in rows), 1),
    "best": rows[0]["plant"],
    "best_cf": rows[0]["capacity_factor_pct"],
    "alerts": len(alerts),
    "alert_plants": ", ".join(alerts) if alerts else "aucune",
}
print(verdict)
notebookutils.notebook.exit(json.dumps(verdict,
ensure_ascii=False))
```

La cellule de paramètres
Le pipeline remplace `day` au lancement. Sans cette étiquette, la valeur envoyée serait ignorée.

regexp_extract
Retrouve `noor_midelt` dans le chemin du fichier : c'est ainsi que chaque ligne retrouve sa centrale.

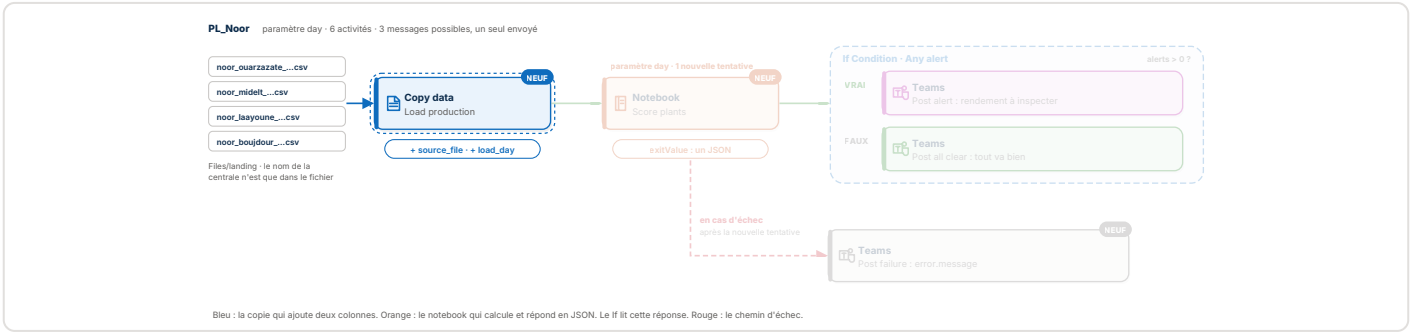
notebook.exit
La dernière ligne rend un texte JSON au pipeline. C'est la « réponse » que le If va lire.

AVANT LE PIPELINE ☐ 4 CSV dans `landing` ☐ `NB_Noor` importé ☐ `LH_Noor` ajouté ☐ cellule de paramètres

ÉTAPE 2

Une copie, quatre fichiers, deux colonnes en plus

1 · OBJETS | **2 · COPIER ET ÉTIQUETER** | 3 · DÉCIDER | 4 · LE CHEMIN D'ÉCHEC



3 PL_Noor et son paramètre

PIPELINE

Nouvel élément, **Pipeline**, nom `PL_Noor`. Canevas vide, onglet **Paramètres**, **+ Nouveau** : nom `day`, type **String**, valeur par défaut `2026-10-21`.

4 Load production

COPY DATA · COLONNES SUPPLÉMENTAIRES

Source, connexion	`LH_Noor`
Racine	`Fichiers`
Type de chemin	Chemin d'accès avec caractères génériques
Dossier	`landing`
Nom de fichier	`*.csv`
Format	DelimitedText, Première ligne comme en-tête coché

Destination, connexion	`LH_Noor`
Racine	`Tables`
Table	Nouvelle : `production_raw`
Action de table	`Remplacer` (Overwrite)

Onglet **Source**, dépliez **Avancé**, **Colonnes supplémentaires**, **+ Nouveau**, deux fois :

NOM	VALEUR	CE QUE LA LIGNE REÇOIT
`source_file`	dans la liste : `$$FILEPATH` (chemin du fichier)	Le chemin du fichier dont elle vient, par exemple … `noor_midelt_2026-10-21.csv`
`load_day`	**Ajouter du contenu dynamique** : `@pipeline().parameters.day`	Le jour traité, le même pour les 96 lignes

Exécution 1 la copie seule

Enregistrer, **Exécuter**, **OK**. Dans `LH_Noor`, **Tables**, **Actualiser**, ouvrez `production_raw` : **96** lignes et **cinq** colonnes, dont `source_file` et `load_day` remplies.

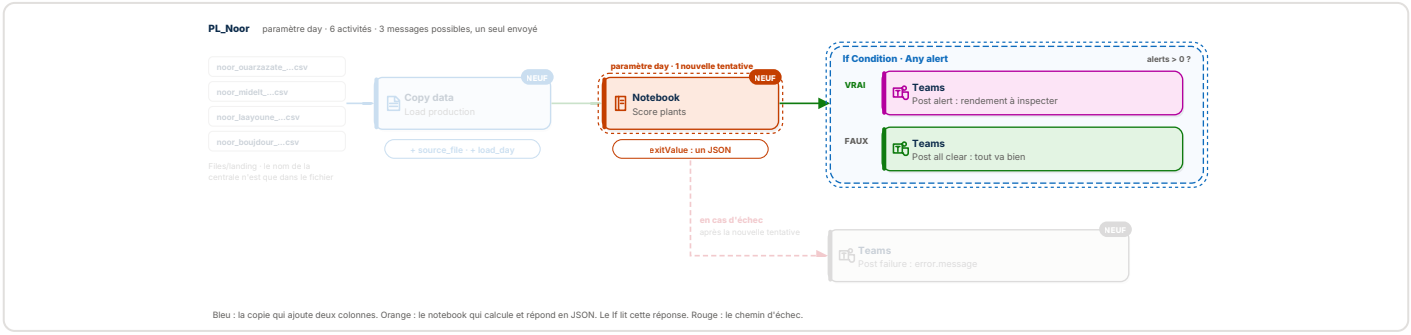
CE QUE J'AIME ICI Sans cette option, il faudrait une copie par centrale ou un ForEach. Ici, une seule copie charge tout le dossier et chaque ligne garde la trace de son fichier : c'est le lignage au niveau de la ligne.

DEMAIN, UNE CINQUIÈME CENTRALE Son CSV arrive dans `landing` : la copie le prend sans rien changer. Il suffit d'ajouter sa capacité dans le dictionnaire `PLANTS` du notebook.

ÉTAPE 3

Le notebook répond, le pipeline décide

1 · OBJETS | 2 · COPIER ET ÉTIQUETER | 3 · DÉCIDER | 4 · LE CHEMIN D'ÉCHEC



5 Score plants — NOTEBOOK PARAMÉTRÉ

1. **Notebook**, nom `Score plants`, relié à la sortie verte de `Load production`.
2. Onglet **Général** : **Nouvelle tentative** `1`, **Intervalle de nouvelle tentative** `30`.
3. Onglet **Paramètres** : votre espace, notebook `NB_Noor`. **Paramètres de base**, **+ Nouveau** : nom `day`, type **String**, valeur :

```
@pipeline().parameters.day
```

6 Any alert — IF CONDITION

1. **Condition Si**, nom `Any alert`, relié à la sortie verte de `Score plants`. **Expression** :

```
@greater(json(activity('Score plants').output.result.exitValue).alerts, 0)
```

2. **Vrai** (crayon) : **Microsoft Teams** `Post alert`. **Faux** : **Microsoft Teams** `Post all clear`. Titre des deux : `Maroc solaire`. Messages : lignes **[Post alert]** et **[Post all clear]** de `expressions.txt`.

LIRE LA RÉPONSE EN TROIS TEMPS

`output.result.exitValue` est le texte rendu par `notebook.exit`. `json(…)` le transforme en objet. `.alerts` lit un champ, comme une colonne.

Exécution 2 — le jour réel

Enregistrer, **Exécuter**, **OK** (deux à trois minutes, Spark démarre). Sortie de `Score plants`, `exitValue` :

```
{"day": "2026-10-21", "plants": 4, "total_mwh": 3369.8, "best": "Noor Ouarzazate", "best_cf": 19.5, "alerts": 1, "alert_plants": "Noor Boujdour"}
```

`alerts` vaut 1 : la branche **Vrai** part. Dans Teams :

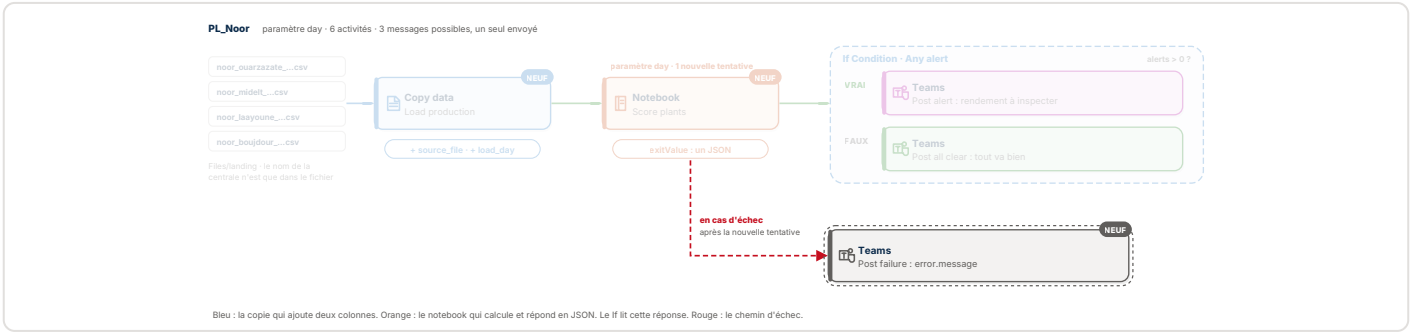


UN SEUL MESSAGE `Post all clear` reste gris : **Ignoré**. Dans un If, une seule branche s'exécute.

ÉTAPE 4

Prévoir la panne : la flèche rouge

1 · OBJETS | 2 · COPIER ET ÉTIQUETER | 3 · DÉCIDER | 4 · LE CHEMIN D'ÉCHEC



7 Post failure

TEAMS SUR ÉCHEC

1. Sur le canevas principal, **Microsoft Teams**, nom `Post failure`, posé sous le If.
2. Survolez `Score plants` : à droite, quatre petites icônes. Tirez la **croix rouge** (**En cas d'échec**) jusqu'à `Post failure`. La flèche est rouge.
3. Titre `Maroc solaire`, message : ligne **[Post failure]** de `expressions.txt`.

```
@{activity('Score plants').error.message}
```

Ce morceau du message lit la cause exacte de l'échec, telle que le notebook l'a levée.

LES QUATRE FLÈCHES Vert : en cas de réussite. Rouge : en cas d'échec. Bleu : à la fin, quoi qu'il arrive. Gris : si l'activité est ignorée. Chaque activité peut en avoir plusieurs.

Exécution 3 — provoquer la panne, exprès

Exécuter. Dans la fenêtre, remplacez `day` par :

```
2026-13-99x
```

La copie réussit. Le notebook refuse la date, attend 30 s, réessaie, échoue encore. La flèche rouge s'allume. Dans Teams :

```
Maroc solaire, 2026-13-99x : le calcul a échoué
malgré une nouvelle tentative. Cause : …
ValueError: Date invalide reçue du pipeline :
2026-13-99x …
```

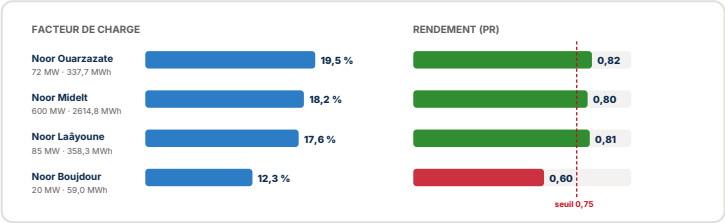
LE PIPELINE RESTE ROUGE C'est voulu : l'échec est signalé, pas caché. Le Moniteur affiche `Score plants` en échec, avec deux tentatives.

REMETTRE EN ORDRE Relancez avec la valeur par défaut `2026-10-21` : la copie remplace les lignes de test, tout redevient vert.

POURQUOI JE PROVOQUE LA PANNE Un chemin d'échec qu'on n'a jamais vu s'allumer est un chemin qui ne marche peut-être pas. Le tester une fois, exprès, c'est la seule façon d'en être sûr.

RÉFÉRENCE

Lire le résultat, retenir, débloquer



3 369,8
MWh produits le 21 octobre

0,60
rendement de Boujdour, sous le seuil de 0,75

19,5 %
facteur de charge de Ouarzazate, en tête

78 %
de l'énergie vient de Midelt seule

Vérifier dans le point de terminaison SQL

```
SELECT plant, capacity_mw, mwh, capacity_factor_pct,
       performance_ratio, status
FROM plant_scores ORDER BY capacity_factor_pct DESC;
```

PLANT	MW	MWH	CF %	PR	STATUS
Noor Ouarzazate	72	337.7	19.5	0.82	ok
Noor Midelt	600	2614.8	18.2	0.80	ok
Noor Laâyoune	85	358.3	17.6	0.81	ok
Noor Boujdour	20	59.0	12.3	**0.60**	**alerte**

Étiqueter à la source
`$$FILEPATH` garde l'origine de chaque ligne : une seule copie pour tout un dossier, et une trace vérifiable.

Un notebook qui répond
`notebook.exit` plus `json()` : le calcul ne se contente plus de réussir, il rend une donnée qui pilote la suite.

Échouer proprement
Une nouvelle tentative pour les incidents passagers, puis un message qui dit pourquoi.

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Notebook : « Table or view not found: production_raw »	Le lakehouse n'est pas attaché à `NB_Noor`. Ouvrez-le, ajoutez `LH_Noor`, enregistrez.
Le notebook calcule toujours le 21 octobre	La cellule de paramètres n'est pas étiquetée. **...**, **Activer/désactiver la cellule de paramètre**.
`plant_scores` vide, `alerts` à 0	`load_day` est vide : la valeur a été tapée comme du texte. **Ajouter du contenu dynamique**, recollez.
Any alert : « exitValue » introuvable	La dernière ligne du notebook n'a pas tourné. Avec un ancien runtime, remplacez `notebookutils` par `mssparkutils`.
Post failure part à chaque exécution	La flèche est bleue (**À la fin**) au lieu de rouge. Supprimez-la, retirez la croix rouge.
Échec « AADSTS700082 »	Connexion Teams expirée. **Gérer les connexions** ou nouvelle connexion, **Compte d'organisation**.